

Batched Eigenvalue Decomposition — PR #172672

The Problem

Computing eigenvalues for batches of matrices (common in PCA, quantum physics, and covariance analysis) used the default cuSolver path—which processes matrices one-by-one even when given a batch. For 16 matrices of size 512×512, this meant 16 sequential cuSolver calls taking 22,692 μs total. GPU utilization: only 6% because each call was small and serial.

The Solution

PR: #172672 - Fast memory snapshot (Merged Jan 30, 2026)

Performance: Up to 3331x faster (726.8ms → 0.2ms for 100k trace entries)

```
# Before: Sequential eigenvalue decomposition
def compute_eigenvalues_batch(matrices): # [batch, N, N]
    eigenvalues = []
    for i in range(matrices.shape[0]):
        # Each matrix processed separately by cusolverDnSsyevd
        # No batching, 16 sequential GPU kernel launches
        vals, vecs = torch.linalg.eigh(matrices[i])
        eigenvalues.append(vals)
    return torch.stack(eigenvalues)
# Time: 22,692 μs for batch=16, GPU utilization ~6%

# After: Batched cuSolver API (syevj_batched)
def compute_eigenvalues_batch(matrices): # [batch, N, N]
    # Single call to cusolverDnSsyevjBatched
    # Processes all 16 matrices in parallel
    vals, vecs = torch.linalg.eigh(matrices)
    return vals

# Internal dispatch (simplified):
if matrices.dim() == 3 and matrices.is_cuda:
    # Use batched cuSolver API
    return linalg_eigh_cusolver_syevj_batched(matrices)
else:
    # Fall back to sequential for CPU or single matrix
    return linalg_eigh_sequential(matrices)

# Time: 969 μs for batch=16 (23x faster)
# GPU utilization: 85% (all matrices processed in parallel)
```

The Pattern

■ When to Apply

- Batched linear algebra operations (SVD, QR, eigenvalues)
- Loop over independent matrices with same operation
- Profile shows sequential small kernel launches

■ How to Apply

- Use batch-aware APIs: `cusolverDn*Batched` functions
- Stack matrices into [batch, M, N] tensor before operation
- Check library support for batched variant
- Add heuristics to dispatch to batched path automatically

■ Profiler Signals

- Loop calling `torch.linalg.*` on individual matrices
- Low GPU utilization despite many operations
- Timeline shows gaps between kernel launches

Triton Kernel Cache Reuse — PR [#181137](#)

The Problem

torch.compile with CUDAGraph partitioning (splits graph into subgraphs for memory efficiency) generated duplicate Triton kernels— each partition compiled identical kernels independently. For Llama 3.1-8B with 10 partitions, this meant compiling the same attention kernel 10 times. Total compilation time: 69.18 seconds, with 42 seconds wasted on duplicates.

The Solution

PR: [#181137](#) - Add TLS stack_bounds on aarch64 (Merged Apr 23, 2026)

Performance: Up to 479x faster in direct mode (187.02μs → 0.39μs per call)

```

# Before: Each partition compiles kernels independently
class CUDAGraphPartitioner:
    def compile_partition(self, subgraph, partition_id):
        # Each partition has its own compilation context
        # Duplicate kernels compiled from scratch
        triton_kernels = []
        for node in subgraph.nodes:
            if is_triton_kernel(node):
                # Compile Triton kernel (50-200ms per kernel)
                kernel = triton.compile(node.kernel_source)
                triton_kernels.append(kernel)
        return triton_kernels

# For 10 partitions with same attention kernel:
# Compile time: 10 × 4.2s = 42 seconds (wasted!)

# After: Share kernel cache across partitions
class CUDAGraphPartitioner:
    def __init__(self):
        # Global kernel cache keyed by source code hash
        self.kernel_source_cache = {}

    def compile_partition(self, subgraph, partition_id):
        triton_kernels = []
        for node in subgraph.nodes:
            if is_triton_kernel(node):
                # Check if we've compiled this kernel before
                source_hash = hash(node.kernel_source)
                if source_hash in self.kernel_source_cache:
                    # Reuse compiled kernel!
                    kernel = self.kernel_source_cache[source_hash]
                else:
                    # Compile once, cache for other partitions
                    kernel = triton.compile(node.kernel_source)
                    self.kernel_source_cache[source_hash] = kernel
                triton_kernels.append(kernel)
        return triton_kernels

# For 10 partitions: compile once, reuse 9 times
# Compile time: 4.2s (1 compile) instead of 42s (10 compiles)
# Llama 3.1-8B: 69.18s → 26.81s (61% reduction)

```

The Pattern

■ When to Apply

- Multiple compilation contexts (partitions, modules, layers)
- Identical or similar kernels compiled repeatedly
- First-run compilation time dominates workflow

■ How to Apply

- Cache compiled kernels by source code hash
- Share cache across compilation boundaries
- Use persistent disk cache for cross-session reuse
- Implement cache invalidation when source changes

■ Profiler Signals

- Long first-run compilation time
- Multiple "Compiling..." messages for same kernel
- Cache directory shows duplicate .ptx files

Horizontal Kernel Fusion (Combo Kernels) — PR [#172106](#)

The Problem

torch.compile generated many small independent Triton kernels—operations that don't depend on each other but run sequentially. A transformer layer might have 20 small kernels ($<10\mu\text{s}$ each): layernorm, dropout, bias adds, activations. Each kernel launch adds 5-10 μs overhead. Total overhead: 100-200 μs per layer, 8ms for an 80-layer model—pure launch latency, no useful compute.

The Solution

PR: [#172106](#) - Fix MPS mul performance regression (Merged Jan 9, 2026)

Performance: 3.9-4.2x faster than regressed version, 20-25% faster than baseline (2.95s $\rightarrow$ 0.71s)

```

# Before: Sequential independent kernels
def transformer_layer(x, weight, bias):
    # Kernel 1: LayerNorm (10µs)
    normalized = F.layer_norm(x, normalized_shape)

    # Kernel 2: Linear (8µs)
    linear_out = F.linear(normalized, weight)

    # Kernel 3: Bias add (5µs)
    biased = linear_out + bias

    # Kernel 4: GELU activation (7µs)
    activated = F.gelu(biased)

    # Kernel 5: Dropout (6µs)
    dropped = F.dropout(activated, p=0.1)

    return dropped
# Total: 36µs compute + 25µs launch overhead (5µs × 5 kernels)

# After: Combo kernel fusion (TorchInductor pass)
@triton.jit
def combo_kernel_transformer_ops(
    x_ptr, weight_ptr, bias_ptr, out_ptr,
    N: tl.constexpr, D: tl.constexpr
):
    idx = tl.program_id(0) * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)

    # Load input once
    x = tl.load(x_ptr + idx)

    # Fuse all ops in registers (no intermediate HBM writes!)
    # 1. LayerNorm
    mean = tl.sum(x) / D
    var = tl.sum((x - mean) ** 2) / D
    normalized = (x - mean) / tl.sqrt(var + 1e-5)

    # 2. Linear (simplified for element-wise)
    weight = tl.load(weight_ptr + idx)
    linear_out = normalized * weight

    # 3. Bias add
    bias = tl.load(bias_ptr + idx)
    biased = linear_out + bias

    # 4. GELU
    activated = biased * 0.5 * (1.0 + tl.erf(biased / 1.41421))

    # 5. Dropout (mask precomputed)
    dropped = activated * dropout_mask

    # Write once
    tl.store(out_ptr + idx, dropped)

# Total: 36µs compute + 5µs launch overhead (1 kernel)
# Speedup: 20µs saved per layer × 80 layers = 1.6ms per forward pass

```

The Pattern

■ When to Apply

- Many small independent operations ($<20\mu\text{s}$ each)
- Operations have no data dependencies (can run in any order)
- Profiler shows high kernel launch overhead

■ How to Apply

- Identify independent ops via topological sort
- Group up to 8 ops per combo kernel (balance size/benefit)
- Generate single kernel combining all operations
- Enable with: `torch._inductor.config.combo_kernel = True`

■ Profiler Signals

- Many $<20\mu\text{s}$ kernel launches in sequence
- Total launch overhead $>10\%$ of execution time
- Operations are element-wise or broadcasted

cuSolver Backend for Eigenvalues — PR [#164789](#)

The Problem

`torch.linalg.eig` (general eigenvalue decomposition) used PyTorch's custom CUDA implementation, which was slower than NVIDIA's cuSolver library. For a 2048×2048 matrix on H100, PyTorch's implementation took 18ms while cuSolver could do it in 1.8ms— a 10x gap. On consumer GPUs (RTX 4070), still 2x slower. Missing out on years of NVIDIA's optimization effort.

The Solution

PR: [#164789](#) - Update operator benchmark baseline (Merged Oct 7, 2025)

Performance: 1.34-1.61x faster across multiple ops (best: `add_M1_N1_K1_cpu` at 1.61x)

```
# Before: PyTorch custom CUDA implementation
def linalg_eig_pytorch(A): # A: [N, N]
    # Custom CUDA kernels for eigenvalue decomposition
    # Implements QR algorithm in PyTorch CUDA
    # Time: 9ms on RTX 4070, 18ms on H100
    eigenvalues, eigenvectors = aten_eig_cuda(A)
    return eigenvalues, eigenvectors

# After: Dispatch to cuSolver when available
def linalg_eig(A):
    if A.is_cuda and has_cusolver_dnxgeev and A.dtype in [float32, float64]:
        # Use cusolverDnXgeev (CUDA 12.8+)
        # Highly optimized by NVIDIA for datacenter GPUs
        eigenvalues, eigenvectors = cusolver_dnxgeev(A)
    else:
        # Fall back to PyTorch implementation
        eigenvalues, eigenvectors = aten_eig_cuda(A)
    return eigenvalues, eigenvectors

# cuSolver implementation leverages:
# - Optimized BLAS/LAPACK routines
# - Tensor core acceleration on Hopper (H100)
# - Better memory access patterns
# Time: 4.5ms on RTX 4070, 1.8ms on H100

# Usage (no code changes needed):
A = torch.randn(2048, 2048, device='cuda')
eigenvalues, eigenvectors = torch.linalg.eig(A)
# Automatically uses cuSolver if available
```

The Pattern

■ When to Apply

- Custom implementation slower than vendor library
- Operation maps cleanly to library API
- Library available on target platforms (CUDA 12.8+)

■ How to Apply

- Profile custom implementation vs vendor library
- Add dispatch logic: check version/availability
- Provide fallback for older CUDA versions
- Test numerical accuracy matches (important for linalg!)

■ Profiler Signals

- Linear algebra ops taking longer than expected
- Vendor library exists for same operation
- Significant speedup on newer hardware (H100 vs A100)

FlexAttention for CPU Inference — PR [#152361](#)

The Problem

FlexAttention (flexible attention patterns: sliding window, causal, block-diagonal) only worked on CUDA. CPU inference fell back to naive PyTorch attention—materializing full $N \times N$ attention matrix in memory. For 8K sequence length, that's 256 MB per head—causing OOM on consumer CPUs. Also missed vectorization opportunities (AVX2/AVX512) available in CPU kernels.

The Solution

PR: #152361 - Build libgomp (gcc-13) on AArch64 (Merged Oct 10, 2025)

Performance: Up to 2x faster on AArch64 (100% speedup at small inputs/high threads)

```
# Before: No CPU support, falls back to naive attention
def flex_attention_cpu(q, k, v, score_mod_fn):
    # FlexAttention not available on CPU
    # Fall back to materializing full attention matrix
    scores = torch.matmul(q, k.transpose(-2, -1)) # [batch, heads, N, N]

    # Apply custom score modification (e.g., sliding window mask)
    scores = score_mod_fn(scores) # Still full N×N matrix!

    attn = F.softmax(scores, dim=-1)
    output = torch.matmul(attn, v)
    return output
# Memory:  $O(N^2)$  = 256 MB for 8K sequence
# No vectorization, no fusion

# After: CPU-optimized FlexAttention via torch.compile
@torch.compile
def flex_attention_cpu(q, k, v, score_mod_fn):
    # torch.compile generates CPP template kernels
    # Supports bf16 and fp32 with AVX2/MKLDNN when available

    # Inductor generates fused kernel:
    # - Computes attention in tiles (no full N×N materialization)
    # - Applies score_mod_fn on-the-fly
    # - Uses AVX2 vectorization (8 floats at once)
    # - Fuses softmax into attention computation

    return torch._C._nn.flex_attention(q, k, v, score_mod_fn)
# Memory:  $O(N)$  = 2 MB for 8K sequence (128x reduction)
# Vectorized with AVX2, fused operations

# Usage:
def sliding_window_mask(score, b, h, q_idx, kv_idx):
    return torch.where(q_idx - kv_idx < 256, score, float('-inf'))

q, k, v = ... # [batch, heads, seq_len, head_dim] on CPU
output = flex_attention(q, k, v, score_mod=sliding_window_mask)
# Works on CPU with torch.compile!
```

The Pattern

■ When to Apply

- CUDA-only operation needed for CPU inference
- CPU has specialized instructions (AVX2, MKLDNN)
- Operation can be tiled to avoid memory explosion

■ How to Apply

- Extend torch.compile CPP template support
- Add CPU kernel dispatch path
- Use tiling to keep memory $O(N)$ instead of $O(N^2)$
- Leverage SIMD instructions via compiler intrinsics

■ Profiler Signals

- CPU inference OOM or very slow vs CUDA
- Fallback to unoptimized path on CPU
- No SIMD instructions in disassembly

Conv+BatchNorm Memory Optimization — PR [#158239](#)

The Problem

Training with Conv+BatchNorm pattern (ubiquitous in ResNet, EfficientNet) stored full feature maps for backward pass. For ResNet50 with batch size 64, layer conv1 outputs $64 \times 64 \times 112 \times 112 = 51$ million elements (204 MB in FP32). Multiply by 50 conv layers = 10 GB just for feature maps. This memory could be used for larger batch sizes or bigger models.

The Solution

PR: [#158239](#) - Fix MPS index_kernel for large tensors (Merged Jul 16, 2025)

Performance: 2.25-2.47x faster for 11x2000x2000 tensors ($4870.4\mu\text{s} \rightarrow 1972.6\mu\text{s}$)

```

# Before: Store feature maps for backward
def conv_bn_forward(x, conv_weight, bn_weight, bn_bias):
    # Convolution
    conv_out = F.conv2d(x, conv_weight) # [64, 64, 112, 112]

    # Save feature map for backward (204 MB!)
    ctx.save_for_backward(conv_out)

    # BatchNorm
    bn_out = F.batch_norm(conv_out, bn_weight, bn_bias)
    return bn_out

def conv_bn_backward(grad_output):
    conv_out = ctx.saved_tensors[0] # Load 204 MB from memory
    # Compute gradients using conv_out
    ...

# Memory: Store conv_out (204 MB) × 50 layers = 10 GB

# After: Weight normalization instead of feature map normalization
def conv_bn_forward(x, conv_weight, bn_weight, bn_bias):
    # Fuse BatchNorm into conv weights (done once at forward)
    # Normalize weights instead of activations
    weight_mean = conv_weight.mean(dim=(1, 2, 3), keepdim=True)
    weight_var = conv_weight.var(dim=(1, 2, 3), keepdim=True)
    normalized_weight = (conv_weight - weight_mean) / torch.sqrt(weight_var + 1e-5)
    scaled_weight = normalized_weight * bn_weight.view(-1, 1, 1, 1)

    # Convolution with normalized weights
    conv_out = F.conv2d(x, scaled_weight, bias=bn_bias)

    # Don't save conv_out! Save normalized weights instead (much smaller)
    ctx.save_for_backward(normalized_weight, x)

    return conv_out

def conv_bn_backward(grad_output):
    # Recompute conv_out from x and normalized_weight if needed
    # Much less memory since we only save weights (64 × 3 × 3 = 576 elements)
    # vs feature maps (51 million elements)
    ...

# Memory: Store weights (2 KB) × 50 layers = 100 KB (vs 10 GB!)
# Trade-off: Slight recomputation in backward, but 100x memory savings

```

The Pattern

■ When to Apply

- Normalization layer after convolution/linear
- Training memory-bound (can't increase batch size)
- Willing to trade small compute for large memory savings

■ How to Apply

- Identify Norm(Conv(x)) patterns
- Normalize weights instead of activations
- Save small tensors (weights) vs large (feature maps)
- Recompute activations in backward if needed

■ Profiler Signals

- OOM during training with moderate batch size
- Large saved tensors in autograd graph
- Conv+BN or Linear+LayerNorm patterns

ARM64 CPU Optimizations (MKLDNN) — PR [#128717](#)

The Problem

ARM-based cloud instances (AWS Graviton, Apple Silicon) ran PyTorch with generic CPU kernels—no specialized optimizations for ARM's NEON SIMD instructions or ARM Compute Library. BERT inference on AWS c7g (Graviton3): 2800 tokens/sec with generic kernels. Meanwhile, x86 with MKLDNN achieved 7500 tokens/sec—2.7x faster just from architecture-specific optimizations.

The Solution

PR: #128717 - Fix compile time regression (Merged Jun 19, 2024)

Performance: Over 2x faster compile time (6+ minutes → 2:48) by caching `get_gpu_type()`

```
# Before: Generic CPU kernels for ARM
# Build PyTorch on ARM without USE_MKLDNN_ACL:
# - Uses Eigen or ATen generic CPU kernels
# - No NEON vectorization
# - No operator fusion (conv+relu separate)
def bert_inference_generic(input_ids):
    embeddings = model.embeddings(input_ids) # Generic matmul
    for layer in model.layers:
        # Each op uses generic kernel (no NEON)
        attn = layer.attention(embeddings)    # Slow matmul
        ffn = layer.feed_forward(attn)        # Slow matmul + GELU
        embeddings = layer.layer_norm(ffn)    # No fusion
    return embeddings
# Throughput: 2800 tokens/sec on c7g

# After: Build with USE_MKLDNN_ACL=1
# Enables ARM Compute Library (ACL) backend for MKLDNN
# - NEON-optimized BLAS (8-way SIMD for FP32)
# - Operator fusion via oneDNN graph API
# - Weight prepacking (layouts optimized for NEON)
def bert_inference_mkldnn(input_ids):
    embeddings = model.embeddings(input_ids) # NEON-optimized matmul
    for layer in model.layers:
        # Fused attention (QKVM in one call)
        attn = layer.attention(embeddings)    # 3x faster

        # Fused FFN (matmul+GELU+matmul)
        ffn = layer.feed_forward(attn)        # 2.5x faster

        # Fused LayerNorm+residual
        embeddings = layer.layer_norm(ffn)    # 1.5x faster
    return embeddings
# Throughput: 16,200 tokens/sec on c7g (5.8x improvement)

# Build command for ARM with MKLDNN:
# USE_MKLDNN_ACL=1 python setup.py install
# Requires: ARM Compute Library (ACL) installed
```

The Pattern

■ When to Apply

- Running on non-x86 architecture (ARM, RISC-V)
- Vendor provides optimized library (ACL, MKL, OpenBLAS)
- Significant performance gap vs x86

■ How to Apply

- Build PyTorch with vendor library flags
- Enable Inductor FX passes for platform
- Test operator fusion support (conv+bn, matmul+gelu)
- Benchmark vs generic build (expect 2-6x gains)

■ Profiler Signals

- Inference much slower than x86 with same core count
- CPU kernels show generic names (no vendor suffix)
- No SIMD instructions in disassembly (objdump -d)

128-bit Vector Optimization (sm90+) — PR [#159699](#)

The Problem

PyTorch CUDA kernels used 32-bit vectorization (vec4) universally—loading 4 floats at once. On Hopper (sm90+), 128-bit vector instructions (vec8) can load 8 floats per instruction. But compiling vec8 for older GPUs (sm75, sm80) increased binary size by 40% and compile time by 2x—bloating the library for features unused on those architectures.

The Solution

PR: #159699 - Measure dispatch overhead (Merged Aug 13, 2025)

Performance: ~12x faster with cache hit vs miss (~80μs vs ~930μs)

```
# Before: vec8 compiled for all architectures
template
__global__ void copy_kernel(T* dst, const T* src, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Use vec8 (128-bit) loads for all GPUs
    using vec8 = Vec; // Loads 8 elements at once
    vec8 data = *reinterpret_cast(&src[idx * 8]);
    *reinterpret_cast(&dst[idx * 8]) = data;
}
// Compiled for: sm75, sm80, sm86, sm90
// Binary size: +40% due to vec8 code for sm75/80 (where it's slower!)
# Compile time: +2x

# After: Conditional vec8 only for sm90+
template
__global__ void copy_kernel(T* dst, const T* src, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    #if __CUDA_ARCH__ >= 900 // sm90+ (Hopper)
        // Use vec8 on Hopper (128-bit loads, twice as efficient)
        using vec_type = Vec;
        constexpr int vec_size = 8;
    #else // sm75, sm80, sm86
        // Use vec4 on older GPUs (64-bit loads)
        using vec_type = Vec;
        constexpr int vec_size = 4;
    #endif

    vec_type data = *reinterpret_cast(&src[idx * vec_size]);
    *reinterpret_cast(&dst[idx * vec_size]) = data;
}

# Compiled code:
# - sm75/80/86: Only vec4 code (smaller binary, faster compile)
# - sm90+: vec8 code (better performance)
# Binary size: Baseline (no bloat for older GPUs)
# Compile time: Baseline
# Hopper performance: 1.8x faster memory bandwidth utilization
```

The Pattern

■ When to Apply

- New GPU architecture adds better instructions
- Optimization hurts compile time/binary size for old GPUs
- Can conditionally compile based on SM version

■ How to Apply

- Use `__CUDA_ARCH__` preprocessor checks
- Provide fallback path for older architectures
- Benchmark both paths to verify improvement
- Document minimum SM version for optimization

■ Profiler Signals

- Compile time increased after optimization
- Binary size bloat (+30% or more)
- Optimization only helps newest GPUs

Multigraph Backend Specialization — PR [#180692](#)

The Problem

torch.compile with dynamic shapes traced a graph for worst-case (largest tensor size), then dispatched to specialized backends at runtime. Each dispatch required re-checking tensor shapes and selecting the right kernel—adding 100-200 μ s overhead per call. For BERT with 12 layers called 100 times/sec, that's 120-240ms of pure dispatch overhead (12-24% of total time).

The Solution

PR: [#180692](#) - Resolve ROCm hipblasLT timeout (Merged Apr 20, 2026)

Performance: ~350x improvement (270+ min timeout → 0.46 min)

```

# Before: Single graph + runtime shape checks
@torch.compile
def bert_layer(x): # x: dynamic shape [batch, seq_len, hidden]
    # Dynamo traces once for symbolic shapes
    # Backend generates code for dynamic shapes
    attn = self.attention(x)
    ffn = self.feed_forward(attn)
    return ffn

# At runtime (called 100 times/sec):
for batch in batches:
    # Each call checks shapes and dispatches
    if batch.shape[1] == 128: # 50µs dispatch overhead
        output = compiled_graph_128(batch)
    elif batch.shape[1] == 256: # 50µs dispatch overhead
        output = compiled_graph_256(batch)
    # ...

# Total overhead: 50µs × 12 layers × 100 calls/sec = 60ms/sec wasted

# After: Multiple specialized graphs with lazy compilation
@torch.compile
def bert_layer(x):
    # Single Dynamo trace (symbolic shapes)
    # Multiple backend compilations (one per specialization)
    attn = self.attention(x)
    ffn = self.feed_forward(attn)
    return ffn

# Multigraph backend creates specialization cache:
# specializations = {
#     (batch=32, seq=128): compiled_fn_32_128, # Compiled on first use
#     (batch=64, seq=256): compiled_fn_64_256, # Compiled on first use
# }

# At runtime (fast path):
for batch in batches:
    shape_key = (batch.shape[0], batch.shape[1])
    if shape_key not in compiled_cache:
        # Compile once (lazy), cache by shape
        compiled_cache[shape_key] = inductor.compile(graph, shape_key)

    # Direct dispatch (no shape checks, just hash lookup)
    output = compiled_cache[shape_key](batch) # 10µs dispatch

# Total overhead: 10µs × 12 layers × 100 calls/sec = 12ms/sec (50% reduction)

```

The Pattern

■ When to Apply

- Dynamic shapes but limited set of actual sizes
- High dispatch overhead (>10% of runtime)
- Willing to cache multiple compiled versions

■ How to Apply

- Use `torch._dynamo.config.specialize_int=True`
- Enable lazy compilation per shape
- Cache compiled functions by shape key
- Monitor cache size (evict LRU if needed)

■ Profiler Signals

- High overhead in compiled function dispatch
- Many shape checks in timeline
- Few distinct shapes in practice (e.g., 3-5)

Dynamic Shapes for Combo Kernels — PR [#150145](#)

The Problem

Combo kernels (horizontal fusion from Module 3) only worked with static shapes—tensors had to have known sizes at compile time. Dynamic shape models (where batch size or sequence length varies) couldn't use combo kernels, missing out on the 20-40% speedup from reduced kernel launch overhead. Had to choose: dynamic shapes OR combo kernel optimization, not both.

The Solution

PR: #150145 - Don't exclude `constant_pad_nd` in prologue fusion (Merged Apr 3, 2025, later reverted)

Performance: 7% speedup for masked linear layers with padding (2048-100 × 4096)

```

# Before: Combo kernels disabled for dynamic shapes
@torch.compile
def transformer_ops(x): # x: [batch, seq_len, hidden] - dynamic dims
    # Inductor detects dynamic shapes
    # Disables combo_kernel optimization

    # Falls back to separate kernels:
    normalized = layer_norm(x) # Kernel 1
    linear = matmul(normalized, w) # Kernel 2
    activated = gelu(linear) # Kernel 3
    # Total: 3 kernel launches + overhead

# After: Combo kernels with dynamic shape support
torch._inductor.config.combo_kernel_foreach_dynamic_shapes = True

@torch.compile
def transformer_ops(x): # x: [batch, seq_len, hidden] - dynamic dims
    # Inductor generates combo kernel with symbolic sizes
    @triton.jit
    def combo_kernel_dynamic(
        x_ptr, w_ptr, out_ptr,
        batch: tl.int32, seq_len: tl.int32, # Runtime args!
        BLOCK_SIZE: tl.constexpr
    ):
        # Compute grid size from runtime dimensions
        pid = tl.program_id(0)
        total_elements = batch * seq_len * hidden
        idx = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)

        mask = idx < total_elements # Runtime check

        # Load
        x = tl.load(x_ptr + idx, mask=mask)

        # Fused ops (same as static)
        normalized = layer_norm_inline(x, hidden)
        linear = matmul_inline(normalized, w_ptr, hidden)
        activated = gelu_inline(linear)

        # Store
        tl.store(out_ptr + idx, activated, mask=mask)

    # Launch with runtime dimensions
    combo_kernel_dynamic[grid](x, w, out, x.shape[0], x.shape[1], x.shape[2])
    # Single kernel launch despite dynamic shapes!

# Trade-off: Small overhead from runtime checks (~2%)
# Benefit: Still get combo kernel fusion (~25% speedup vs separate kernels)

```

The Pattern

■ When to Apply

- Dynamic shapes prevent optimization (combo kernels, fusion)
- Shape varies but optimization logic doesn't depend on size
- Willing to accept small runtime overhead (~2%) for 20-40% gain

■ How to Apply

- Pass symbolic dimensions as runtime kernel arguments
- Use runtime masking for boundary conditions
- Compute grid size dynamically (not constexpr)
- Enable via config flag (default off for safety)

■ Profiler Signals

- Optimization disabled due to dynamic shapes
- Static shape version much faster than dynamic
- Shape varies but within reasonable range (not exponential)